



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN

GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09-2

NOMBRE COMPLETO: Miranda González José Francisco

N° de Cuenta: 318222327

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 04

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: 27/10/24

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Reporte de Práctica 9-2: Animación Avanzada

Instrucciones:

1.- Hacer que en el arco que crearon se muestre la palabra: PROYECTO CGEIHG MONOPOLY. animado desplazándose las letras de izquierda a derecha como si fuera letrero LCD/LED de forma cíclica

2.-Separar las cabezas del Dragón y agregar las siguientes animaciones:

- Movimiento del cuerpo ida y vuelta.
- Aleteo
- Cada cabeza se mueve de forma diferente de acuerdo a una función/algoritmo diferente (ejemplos: espiral de Arquímedes, movimiento senoidal, lemniscata, etc...)
- Cada cabeza debe de verse de un color diferente: roja, azul, verde, blanco, café.

Extra 5 puntos: Hacer que cada cabeza escupa/emita su aliento (por medio de texturas con translucidez) al presionar una tecla:

- Roja: Fuego
- Azul: Rayos
- Verde: Gas Venenoso
- Blanco: Hielo
- Café: Arena

Actividades realizadas

Para completar las actividades solicitadas realice lo siguiente:

Para la actividad 1:

Cree la textura con el mensaje solicitado, esta fue de 5X5:



tipografiaLetrero

Dentro del main.cpp los cambios nuevos fueron los siguientes (omitiendo toda la explicación de la creación del arco):

Creación textura.

```
113 // PRACTICA 09-2
114 // MENSAJE: PROYECTO CGEIHG MONOPOLY
115 Texture tipografia;
```

En CreateObjects(), cree los vértices e índices de la textura.

```
283 // PRACTICA 09-2
284 // MENSAJE : PROYECTO CGEIHG MONOPOLY
285 unsigned int mensajeLetreroIndices[] = {
286     0, 1, 2,
287     1, 2, 3,
288 };
289
290 GLfloat mensajeLetreroVertices[] = {
291     5.0f, 5.0f, 0.0f,    0.97f, 1.0f,    0.0f, 0.0f, -1.0f,
292     5.0f, 4.0f, 0.0f,    0.97f, 0.8f,    0.0f, 0.0f, -1.0f,
293     6.0f, 5.0f, 0.0f,    1.0f, 1.0f,    0.0f, 0.0f, -1.0f,
294     6.0f, 4.0f, 0.0f,    1.0f, 0.8f,    0.0f, 0.0f, -1.0f,
295 };
```

```
326 // PRACTICA 09-2
327 // MENSAJE : PROYECTO CGEIHG MONOPOLY
328 Mesh* obj8 = new Mesh();
329 obj8->CreateMesh(mensajeLetreroVertices, mensajeLetreroIndices, 32, 6);
330 meshList.push_back(obj8);
```

Carga de la textura.

```
372 // PRACTICA 09-2
373 // TEXTURA DEL MENSAJE
374 tipografia = Texture("Textures/tipografiaLetrero.png");
375 tipografia.LoadTextureA();
```

Dibujo de arco y letrero.

```
625 // ARCO
626 model = glm::mat4(1.0);
627 model = glm::translate(model, glm::vec3(0.0f, 6.8f, -20.0));
628
629 modelauxArco = model; // GUARDAR
630
631 model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
632 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
633 arco.RenderModel();
634
635 model = modelauxArco; // A PARTIR DEL ARCO
636
637 // LETRERO
638 model = glm::translate(model, glm::vec3(0.0f, 4.0f, 0.4));
639 model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
640 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
641 arcoLetrero.RenderModel();
```

Textura.

```
1025 // PRACTICA 09-2
1026 // IR RECORRIENDO LA TEXTURA DEL MENSAJE
1027 toffsetnumerouLetrero -= 0.0001;
1028 if (toffsetnumerouLetrero <= 0.0 && toffsetnumerovLetrero >= 1.0)
1029 {
1030     toffsetnumerouLetrero = 1.0;
1031     toffsetnumerovLetrero = 0.2;
1032 }
1033 else if (toffsetnumerouLetrero <= 0.0)
1034 {
1035     toffsetnumerouLetrero = 1.0;
1036     toffsetnumerovLetrero += 0.2;
1037 }
1038
1039 toffset = glm::vec2(toffsetnumerouLetrero, toffsetnumerovLetrero);
1040 model = glm::mat4(1.0);
1041 model = glm::translate(model, glm::vec3(-5.5f, 6.3f, -19.0));
1042 glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
1043 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1044 color = glm::vec3(1.0f, 0.0f, 0.0f);
1045 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
1046 tipografia.UseTexture();
1047 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
1048 meshList[7]->RenderMesh();
```

```
66 // ANIMACIÓN DEL LETRERO
67 float toffsetnumerouLetrero = 1.0f;
68 float toffsetnumerovLetrero = 0.2f;
```

En esta parte la idea fue recorrer la textura desde el final hasta el inicio, para que diera el efecto de que las letras se desplazan de izquierda a derecha.



tipografiaLetrero

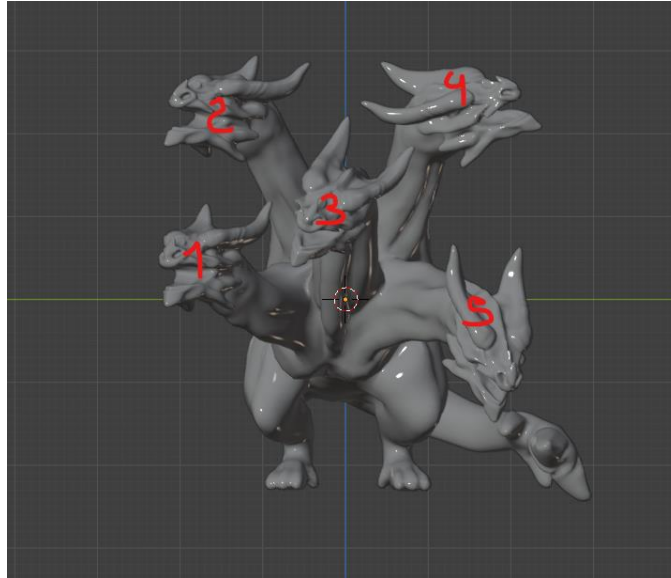
Cada que se llega al final de una fila se da un salto a la fila de arriba hasta recorrerla por completo.

Cuando se llega a la letra “P” del inicio todo se reinicia y volvemos a comenzar en la letra “Y”, esto hace que veamos el mensaje de forma infinita.

Para la actividad 2:

Lo primero que tuve que hacer fue separar las cabezas del dragón e importarlas por separado para luego por medio de jerarquía unir las al cuerpo del dragón.

Para no confundirme con el orden, tome como referencia la siguiente imagen.



Los cambios dentro del main.cpp fueron los siguientes.

Las variables para las animaciones del dragón.

```

70 // ANIMACION DE LAS ALAS
71 float rotaAla01 = 0.0f;
72 float rotaAla01Offset = 0.8f;
73 float rotaAla02 = 0.0f;
74 float rotaAla02Offset = 0.8f;
75 bool bajasube01 = true;
76
77 // ANIMACION DEL DRAGON (TRASLACIONES)
78 float mueveDragon = 0.0f;
79 float mueveDragonOffset = 0.1f;
80 float mueveDragon2 = 0.0f;
81 float mueveDragonOffset2 = 0.1f;
82 float mueveDragon3 = 0.0f;
83 float mueveDragonOffset3 = 0.1f;
84 float mueveDragon4 = 0.0f;
85 float mueveDragonOffset4 = 0.1f;
86
87 // ANIMACIÓN DEL DRAGON (CURVAS)
88 float rotaDragon2 = 0.0f;
89 float rotaDragonOffset2 = 0.45f; // ESTE VALOR ME ASEGURA QUE AL TERMINAR DE MOVERSE POR EL EJE Z LA ROTACION SEA DE 180°
90 float rotaDragon3 = 0.0f;
91 float rotaDragonOffset3 = 0.45f; // ESTE VALOR ME ASEGURA QUE AL TERMINAR DE MOVERSE POR EL EJE Z LA ROTACION SEA DE 180°
92
93 // FUNCION SENO PARA LA CURVA DEL DRAGON
94 float angulovaria2 = 0.0f;
95 float angulovaria3 = 0.0f;

```

Creación de modelos.

```

122 // EJERCICIO CLASE
123 // EJERCICIO 03
124 Model DragonCuerpo;
125 Model DragonAla01;
126 Model DragonAla02;
127
128 // PRACTICA 09-2
129 // ARCO, LETRERO Y DRAGON
130 Model arco;
131 Model arcoLetrero;
132 Model DragonCabeza01;
133 Model DragonCabeza02;
134 Model DragonCabeza03;
135 Model DragonCabeza04;
136 Model DragonCabeza05;

```

Carga de los modelos.

```
384 // EJERCICIO CLASE
385 // EJERCICIO 03
386 DragonCuerpo = Model();
387 DragonCuerpo.LoadModel("Models/cuerpoDragon.obj");
388 DragonAla01 = Model();
389 DragonAla01.LoadModel("Models/ala01Dragon.obj");
390 DragonAla02 = Model();
391 DragonAla02.LoadModel("Models/ala02Dragon.obj");
392
393 // PRACTICA 09-2
394 // ARCO, LETRERO Y DRAGON
395 arco = Model();
396 arco.LoadModel("Models/arco.obj");
397 arcoLetrero = Model();
398 arcoLetrero.LoadModel("Models/letreroArco.obj");
399 DragonCabeza01 = Model();
400 DragonCabeza01.LoadModel("Models/cabeza01Dragon.obj");
401 DragonCabeza02 = Model();
402 DragonCabeza02.LoadModel("Models/cabeza02Dragon.obj");
403 DragonCabeza03 = Model();
404 DragonCabeza03.LoadModel("Models/cabeza03Dragon.obj");
405 DragonCabeza04 = Model();
406 DragonCabeza04.LoadModel("Models/cabeza04Dragon.obj");
407 DragonCabeza05 = Model();
408 DragonCabeza05.LoadModel("Models/cabeza05Dragon.obj");
```

La parte de las animaciones.

Aleteo.

```
488 // PRACTICA 09-2
489 // ALETEO ALA 01 y ALA 02
490 // LLEGAN A 45° Y -45° Y REPITEN
491 if (bajasube01)
492 {
493     if (rotaAla01 < 45.0f && rotaAla02 > -45.0f)
494     {
495         rotaAla01 += rotaAla01offset * deltaTime;
496         rotaAla02 -= rotaAla02offset * deltaTime;
497     }
498     else
499     {
500         bajasube01 = !bajasube01;
501     }
502 }
503 else
504 {
505     if (rotaAla01 > -45.0f && rotaAla02 < 45.0f)
506     {
507         rotaAla01 -= rotaAla01offset * deltaTime;
508         rotaAla02 += rotaAla02offset * deltaTime;
509     }
510     else
511     {
512         bajasube01 = !bajasube01;
513     }
514 }
```

La lógica fue similar al ejemplo del coche que avanzaba y retrocedía de forma infinita.

Movimiento de ida y vuelta.

```
516 // PRACTICA 89-2
517 // IDA Y REGRESO DEL DRAGON
518 if (mueveDragon > -100.0)
519 {
520     mueveDragon -= mueveDragonOffset * deltaTime;
521 }
522 else
523 {
524     if (mueveDragon2 < 40.0)
525     {
526         mueveDragon2 += mueveDragonOffset * deltaTime;
527         // VALORES POSITIVOS PARA EL ANGULO
528         angulovaria2 += 0.5f * deltaTime;
529         rotaDragon2 += rotaDragonOffset2 * deltaTime;
530         //printf("GRADOS AL FINAL : %f", rotaDragon2); // SON 180 GRADOS AL TERMINAR DE DAR LA CURVA, ASI EL DRAGON YA PODRA IR DE REGRESO
531     }
532     else
533     {
534         if (mueveDragon3 > -100.0)
535         {
536             mueveDragon3 -= mueveDragonOffset3 * deltaTime;
537         }
538         else
539         {
540             if (mueveDragon4 < 40.0)
541             {
542                 mueveDragon4 += mueveDragonOffset4 * deltaTime;
543                 // VALORES POSITIVOS PARA EL ANGULO
544                 angulovaria3 += 0.5f * deltaTime;
545                 rotaDragon3 += rotaDragonOffset3 * deltaTime;
546                 //printf("GRADOS AL FINAL : %f", rotaDragon3); // SON 180 GRADOS AL TERMINAR DE DAR LA CURVA, ASI EL DRAGON YA PODRA IR DE REGRESO
547             }
548             else
549             {
550                 // REINICIAR LOS VALORES PARA VOLVER A DAR LA VUELTA
551                 mueveDragon = 0.0f;
552                 mueveDragon2 = 0.0f;
553                 mueveDragon3 = 0.0f;
554                 mueveDragon4 = 0.0f;
555
556                 angulovaria2 = 0.0f;
557                 angulovaria3 = 0.0f;
558
559                 rotaDragon2 = 0.0f;
560                 rotaDragon3 = 0.0f;
561             }
562         }
563     }
564 }
```

En esta parte, al llegar a una cierta distancia sobre el eje -x, el dragón hace una curva representada por una media onda de la función seno mientras avanza por el eje +z, además, durante esa curva el dragón hace una rotación para que al llegar a cierta distancia del eje +z el dragón ya haya rotado 180° en y, y de esta forma ya este posicionado para regresar.

Esta idea se repite para que el dragón vaya de ida y de regreso.

Dibujo del dragón.

```
680 // DRAGON
681 // EJERCICIO PRACTICA 9-2
682 model = glm::mat4(1.0);
683
684 // model = glm::translate(model, glm::vec3(0.0f, 5.0f+sin(glm::radians(angulovaria)), 6.0));
685 // model = glm::translate(model, glm::vec3(0.0f - (angulovaria / 6), 10.0f + (5 * sin(glm::radians(angulovaria * 1))), 6.0));
686 // model = glm::translate(model, glm::vec3(0.0f, 10.0f, 6.0));
687
688 // EL PRIMER TRASLATE (HASTA LLEGAR A UNA PARTE)
689 model = glm::translate(model, glm::vec3(mueveDragon, 10.0f + (5 * sin(glm::radians(angulovaria * 1))), 6.0));
690
691 // SEGUNDO TRASLATE Y ROTATE, AL COMPLETAR EL PRIMERO (PARA HACER UNA CURVA)
692 // EL VALOR NEGATIVO ES PORQUE LA FUNCION SENOIDAL COMIENZA HACIA LOS VALORES POSITIVOS (PARA X), PERO AL QUERER QUE ENPIECE HACIA VALORES NEGATIVOS (PARA -X) SE PONE
693 // EL SIGNO NEGATIVO
694 model = glm::translate(model, glm::vec3((-20 * sin(glm::radians(angulovaria2 * 0.83))), 0.0f, mueveDragon2));
695 model = glm::rotate(model, rotaDragon2 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
696
697 // TERCER TRASLATE, AL COMPLETAR EL SEGUNDO
698 model = glm::translate(model, glm::vec3(mueveDragon3, 0.0f, 0.0f));
699
700 // CUARTO TRASLATE Y ROTATE, AL COMPLETAR EL TERCERO (PARA HACER UNA CURVA)
701 // EL VALOR NEGATIVO ES PORQUE LA FUNCION SENOIDAL COMIENZA HACIA LOS VALORES POSITIVOS (PARA X), PERO AL QUERER QUE ENPIECE HACIA VALORES NEGATIVOS (PARA -X) SE PONE
702 // EL SIGNO NEGATIVO
703 model = glm::translate(model, glm::vec3((-20 * sin(glm::radians(angulovaria3 * 0.83))), 0.0f, mueveDragon4));
704 model = glm::rotate(model, rotaDragon3 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
705
706 modelauxDragon = model;
707
708 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
709 // model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
710 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
711 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
712 DragonCuerpo.RenderModel();
713
714 model = modelauxDragon; // A PARTIR DEL DRAGON
```

En esta parte se guardan todas las transformaciones en la matriz auxiliar, las cuales pertenecen a la animación.

La primera representa como el dragón va hacia el eje -x

La segunda representa la curva que hace para regresar.

La tercera representa como el dragón va hacia el eje x.

La cuarta y última representa la curva que hace para regresar a su punto de partida y de esta forma volver a iniciar la animación.

Alas del dragón.

```
725
726     model = glm::translate(model, glm::vec3(0.3f, 0.21f, -0.26));
727     model = glm::rotate(model, rotaAla01 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
728     model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
729     Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
730     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
731     DragonAla01.RenderModel();
732
733     model = modelauxDragon; // A PARTIR DEL DRAGON
734
735     model = glm::translate(model, glm::vec3(0.18f, 0.52f, 0.4));
736     model = glm::rotate(model, rotaAla02 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
737     model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
738     Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
739     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
740     DragonAla02.RenderModel();
741
742     model = modelauxDragon; // A PARTIR DEL DRAGON
```

Cabezas del dragón.

```
743
744     // TRANSLATE ORIGINAL
745     // model = glm::translate(model, glm::vec3(-1.0f, -0.5f, -0.7));
746     // TRANSLATE DE LA LEMNISCATA, MANTINIENDO LOS VALORES ORIGINALES DEL TRANSLATE Y AGREGANDO LAS FUNCIONES PARA LA LEMNISCATA
747     // EL 0.5 ES EL TAMAÑO DE LA LEMNISCATA, EN ESTE CASO ES PEQUEÑA
748     // Y AL MULTIPLICAR POR 0.1 ES PARA QUE HAGA LA ANIMACION LO MAS CERCA POSIBLE DE SU PUNTO ORIGINAL
749     // AL DIVIDIR ENTRE 7 HAGO QUE SEA MAS LENTA LA ANIMACION
750     model = glm::translate(model, glm::vec3(-1.0f,
751                                             -0.5f + (0.5 * sin(angulovaria/7)) * cos(angulovaria/7)) / (0.5 + sin(angulovaria/7)) * sin(angulovaria/7)) * 0.1,
752                                             -0.7f + (0.5 * cos(angulovaria / 7)) / (0.5 + sin(angulovaria / 7)) * sin(angulovaria / 7)) * 0.1));
753     model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
754     Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
755     color = glm::vec3(1.0f, 0.0f, 0.0f);
756     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
757     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
758     DragonCabeza01.RenderModel();
759
760     model = modelauxDragon; // A PARTIR DEL DRAGON
761
762     // TRANSLATE ORIGINAL
763     // model = glm::translate(model, glm::vec3(-0.83f, 0.6f, -0.5));
764     // TRANSLATE DE LA FUNCION SENOIDAL (LA MISMA QUE TIENE EL DRAGON), MANTINIENDO LOS VALORES ORIGINALES DEL TRANSLATE Y AGREGANDO LAS FUNCION SENOIDAL
765     // 0.1 * PARA LA AMPLITUD
766     // * 5 PARA LONGITUD DE ONDA
767     model = glm::translate(model, glm::vec3(-0.83f,
768                                             0.6f + (0.1 * sin(glm::radians(angulovaria * 5))),
769                                             -0.5));
770     model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
771     Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
772     color = glm::vec3(0.0f, 0.0f, 1.0f);
773     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
774     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
775     DragonCabeza02.RenderModel();
776
777     model = modelauxDragon; // A PARTIR DEL DRAGON
```

```

778 // TRANSLATE ORIGINAL
779 // model = glm::translate(model, glm::vec3(-1.5f, -0.14f, 0.0));
780 // TRANSLATE DEL ESPIRAL DE ARQUIMEDES, MANTINIEDNO LOS VALORES ORIGINALES DEL TRANSLATE Y AGREGANDO EL ESPIRAL DE ARQUIMEDES
781 // 0.001f y 0.01f: ESTOS VALORES CONTROLAN LA FORMA DEL ESPIRAL. 0.001 PARA DEFINIR EL RADIO INICIAL y 0.01 PARA CONTROLAR CUANTO CRECE EL RADIO CADA VUELTA.
782 model = glm::translate(model, glm::vec3(-1.5f,
783 -0.14f + {(0.001f + 0.01f * glm::radians(angulovaria)) * sin(glm::radians(angulovaria))},
784 0.0f + {(0.001f + 0.01f * glm::radians(angulovaria)) * cos(glm::radians(angulovaria))}));
785
786 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
787 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
788 color = glm::vec3(0.0f, 1.0f, 0.0f);
789 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
790 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
791 DragonCabeza03.RenderModel();
792
793 model = modelauxDragon; // A PARTIR DEL DRAGON
794
795 // TRANSLATE ORIGINAL
796 // model = glm::translate(model, glm::vec3(-0.9f, 0.57f, 0.52));
797 // TRANSLATE DEL MOVIMIENTO CIRCULAR, MANTINIEDNO LOS VALORES ORIGINALES DEL TRANSLATE Y AGREGANDO EL MOVIMIENTO CIRCULAR
798 // 0.1 PARA EL RADIO Y + 10 PARA HACER MAS RAPIDO EL GIRO
799 model = glm::translate(model, glm::vec3(-0.9f, 0.57f + (0.1 * sin(glm::radians(angulovaria * 10))), 0.52f + (0.1 * cos(glm::radians(angulovaria * 10))));
800
801 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
802 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
803 color = glm::vec3(1.0f, 1.0f, 1.0f);
804 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
805 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
806 DragonCabeza04.RenderModel();
807
808 model = modelauxDragon; // A PARTIR DEL DRAGON
809
810 // TRANSLATE ORIGINAL
811 // model = glm::translate(model, glm::vec3(-1.25f, -0.55f, 0.7));
812 // TRANSLATE DEL MOVIMIENTO ROBOTE, MANTINIEDNO LOS VALORES ORIGINALES DEL TRANSLATE Y AGREGANDO EL MOVIMIENTO REBOTE
813 // 0.1 PARA UN REBOTE BAJO
814 model = glm::translate(model, glm::vec3(-1.25f,
815 -0.55f + (abs(0.1f * sin(glm::radians(angulovaria)))),
816 0.7));
817
818 model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
819 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
820 color = glm::vec3(0.478f, 0.255f, 0.067f);
821 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
822 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
823 DragonCabeza05.RenderModel();

```

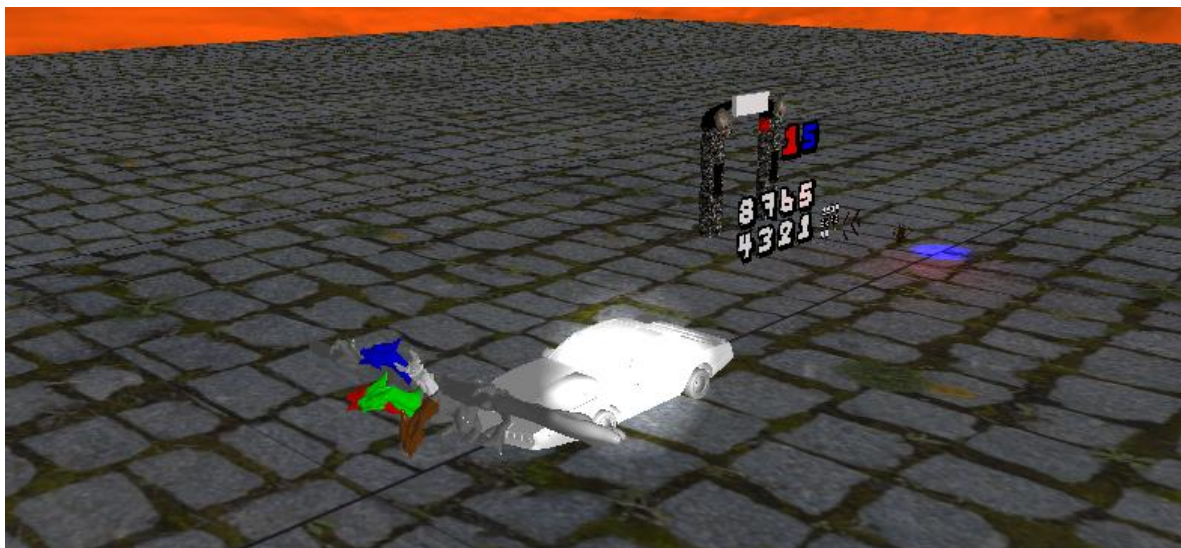
En esta parte les puse color a las cabezas y agregué su movimiento por medio de diferentes funciones.

Busque en internet como implementar las siguientes funciones: LEMNISCATA, FUNCION SENOIDAL, ESPIRAL DE ARQUIMEDES, MOVIMIENTO CIRCULAR y MOVIMIENTO ROBOTE

Después solo fue cuestión de ubicar en que ejes colocarlas y ajustar su velocidad y alcance.

Ejecución del programa:

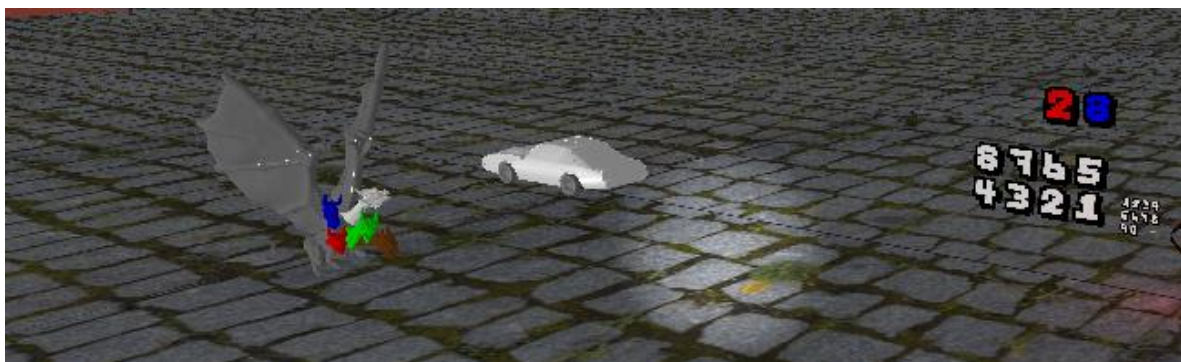
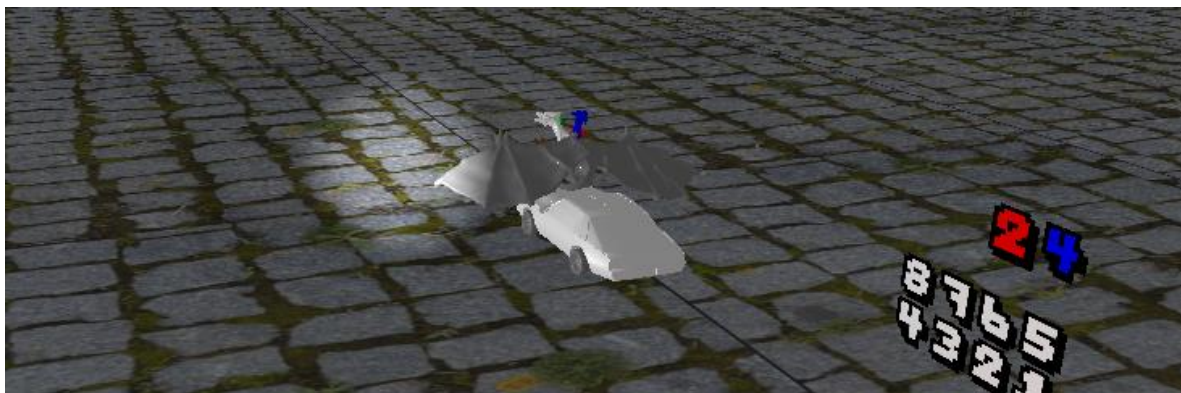
Vista inicial:



Mensaje en el letrero:



Dragón:



Las animaciones no se pueden apreciar bien por medio de las imágenes.

En el video se podrá observar de manera correcta.

Problemas presentados

1. Mensaje

Como se puede ver en el video y en la imagen de la ejecución el mensaje solo se va mostrando letra por letra y esto es porque al crear los índices y vértices elegí unas coordenadas UV muy reducidas (poco menos del tamaño de una letra), esto fue porque al elegir un tamaño más grande (como el de una fila completa) el mensaje se duplicaba.

Por ejemplo, en la ultima fila " P O L Y ", para que las letras fueran de izquierda a derecha comenzaba a recorrer desde la derecha (Y) y llega a la izquierda (P), pero en lo que llegaba a la letra P atrás de esta ya se mostraba otra vez las letras "P O L Y " y eso no lo pude solucionar, porque era necesario que se llegara al final para que diera el efecto de que las letras estaban avanzando, entonces esa fue la razón de hacer mas pequeño el tamaño, pues al llegar por ejemplo a la letra P ya no se alcanzaba a mostrar el mensaje duplicado.

2. Jerarquía

Intente aplicar jerarquía del letrero a la textura como lo hacia con los modelos, pero esto no funciona. Entonces solo acomode la textura (con translate) lo mas cerca del letrero para que diera la impresión de que esta pegada a él.

3. Modelo dragón

Realmente aquí no hubo problemas, pero si fue muy tardado separar las cabezas del cuerpo y después unirlas por medio de jerarquía, pues tenían que quedar bien pegadas al cuerpo junto con las alas.

4. Ida y vuelta

En un principio, cuando el dragón llegaba a cierta distancia del eje -x empezaba a hacer una rotación con la intención de que al llegar a un punto establecido ya estuviera rotado 90° y así pudiera continuar, pero el problema con esto es que realmente no daba el efecto de una curva. Esto lo solucione añadiendo la función seno, pues al llegar al punto establecido daba una media onda mientras que al mismo tiempo rotaba y se trasladaba dando el efecto de una curva.

Conclusión

El ejercicio que mas problemas me dio fue el del letrero, pues no pude encontrar la forma de hacer que no se duplicara el mensaje.

Pensé en otras soluciones como hacer una textura mucho mas grande (25X25) para que todo el mensaje estuviera en una sola fila y así no hubiera problema con repetir el mensaje, pues esa sería la

intención. También pensé en agregar la textura al modelo y ahí agregarle la animación, pero no recordaba cómo se había realizado en clase entonces descarté esa idea.

Al final el mensaje se muestra completo, pero no de la forma que hubiera querido.

Bibliografía

Font Meme. (s.f). Mortal Kombat. Consultado el 26 de octubre del 2024 de <https://fontmeme.com/fuentes/fuente-mortal-kombat/>